# Lab Sheet – 5: Ansible Basics

## Aim

To install Ansible, create an inventory file, and execute basic ad-hoc commands on Linux .

---

## Requirements

- OS: Linux
- Internet connection
- Terminal access
- Basic Linux commands

---

## Level 1: Installation of Ansible

### Step 1: Update System

sudo apt update

### Step 2: Install Ansible

sudo apt install ansible -y

### Step 3: Verify Installation

ansible --version

 Expected Output: Shows installed version of Ansible

---

## Level 2a: Create a Basic Inventory File

### Step 1: Create Inventory File

**nano inventory.ini**

### Step 2: Add Hosts

[local]

localhost ansible_connection=local
 This defines a group named **local** with your system.

**Step 3: Save File**

- Press CTRL + O → Enter
- Press CTRL + X

---

**Level 2b: Run First Ad-Hoc Ansible Command**

**Command: Ping Localhost**

**ansible -i inventory.ini local -m ping**
 Output:

```
localhost | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

## Lab Sheet 6: Ansible Archive

---

**Aim:**
To perform file and directory compression using Ansible modules in Linux .

---

**Requirements:**
Operating System: Linux
Tool: Ansible
Terminal access

---

**Level 1a: Compressing Directory using tar and gz**

**Objective:**
To compress a directory into tar and gz format

**Steps:**

**Step 1:** Create a sample directory
mkdir sampledir
touch sampledir/file1.txt sampledir/file2.txt

**Step 2:** Run Ansible command
ansible -i inventory.ini local -m archive -a "path=sampledir dest=sampledir.tar.gz format=gz"

**Output:**
localhost CHANGED
changed true
dest sampledir.tar.gz

**Explanation:**
archive module is used for compression
format gz creates gzip compressed file

---

**Level 1b: Compress File and Remove Source**

**Objective:**
To compress a file and delete original file

**Steps:**

**Step 1:** Create a file
touch fileA.txt

**Step 2:** Run Ansible command
ansible -i inventory.ini local -m archive -a "path=fileA.txt dest=fileA.tar.gz remove=true format=gz"

**Step 3:** Verify
ls

**Output:**
localhost CHANGED
changed true
dest fileA.tar.gz

**Verification Output:**
fileA.tar.gz

**Explanation:**
remove true deletes original file after compression

---

**Level 2a: Create ZIP Archive for File and Directory**

**Objective:**
To create zip archive

**Steps:**

**Step 1:** Create files and directory
mkdir mydata
touch mydata/a.txt mydata/b.txt

**Step 2:** Run Ansible command
ansible -i inventory.ini local -m archive -a "path=mydata dest=mydata.zip format=zip"

**Step 3:** Verify
ls

**Output:**
localhost CHANGED
changed true
dest mydata.zip

**Verification Output:**
mydata.zip

**Explanation:**
format zip creates zip archive

---

**Level 2b: Create BZIP Archive for File and Directory**

**Objective:**
To create bzip archive

**Steps:**

**Step 1:** Create file
touch fileB.txt

**Step 2:** Run Ansible command
ansible -i inventory.ini local -m archive -a "path=fileB.txt dest=fileB.tar.bz2 format=bz2"

**Step 3:** Verify
ls

**Output:**
localhost CHANGED
changed true
dest fileB.tar.bz2

**Verification Output:**
fileB.tar.bz2

**Explanation:**
format bz2 creates bzip compressed file

---

**Result:**
All archive operations tar gz zip and bzip were successfully performed using Ansible on Linux

## Lab Sheet 7: Ansible Shell Module ADHOC and Playbook

---

**Aim:**
To understand and execute commands using Ansible shell module in adhoc mode and playbook

---

**Requirements:**
Operating System: Linux
Tool: Ansible
Inventory file

---

### Level 1a: Quick Syntax of Ansible Shell Module in Adhoc

**Objective:**
To learn basic syntax of shell module

**Syntax:**
ansible -i inventory.ini group_name -m shell -a "command"

**Example:**
ansible -i inventory.ini local -m shell -a "date"

**Output:**
localhost CHANGED
Sun Apr 19 10:30:00 IST 2026

**Explanation:**
shell module runs commands on remote machine

---

**Level 1b: Ansible Shell in Playbook**

**Objective:**
To execute shell command using playbook

**Steps:**

**Step 1:** Create playbook file
nano shell.yml

**Step 2:** Add content

- - name: Run shell command playbook
  hosts: local
  gather_facts: no

  tasks:
  - name: Display current date
  shell: date

  - name: Show current user
  shell: whoami

  - name: List files
  shell: ls

**Step 3:** Run playbook
ansible-playbook -i inventory.ini shell.yml

**Output:**
TASK display date
changed true

**Explanation:**
Playbook is used for automation of multiple tasks

---

## Level 2a: Execute Single Command using Shell

**Objective:**
To execute a single command

**Steps:**

**Step 1:** Run command
ansible -i inventory.ini local -m shell -a "whoami"

**Output:**
localhost CHANGED
user

**Explanation:**
whoami shows current user

---

## Level 2b: Execute Command with Pipe and Redirection

**Objective:**
To execute commands using pipe and redirection

**Steps:**

**Step 1:** Run command with pipe

```
ansible -i inventory.ini local -m shell -a "ls -l | grep txt"
```

**Step 2:** Run command with redirection

```
ansible -i inventory.ini local -m shell -a "ls > output.txt"
```

**Step 3:** Verify

```
cat output.txt
```

**Output:**

```
localhost CHANGED
file1.txt
file2.txt
```

**Explanation:**

pipe is used to filter output

redirection is used to store output in file

---

**Result:**

Ansible shell module was successfully used in adhoc commands and playbook for executing commands

## Lab Sheet 8: Ansible Playbook Execution and File Management

---

**Aim:**

To run Ansible playbooks and create files and multiple directories using tasks and loops

---

**Requirements:**
Operating System: Linux
Tool: Ansible
Inventory file with groups

---

**Level 1: Run Playbook**

**Objective:**
To execute a basic Ansible playbook

**Playbook File:** nano test.yml

```
- name: Run simple playbook
  hosts: local
  gather_facts: no
  tasks:
    - name: Display message
      shell: echo Hello World
```

**Run Command:**
ansible-playbook -i inventory.ini test.yml

**Output:**
TASK Display message
changed true
Hello World

**Explanation:**
Playbook runs a simple shell command on localhost

---

**Level 2a: Create File on Target Machines**

**Objective:**
To create a file on target systems defined in inventory and webserver group

**Inventory Example:nano inventory.ini**

```
[webserver]
localhost ansible_connection=local
```

**Playbook File:** create_file.yml

```
- name: Create file on servers
  hosts: webserver
  gather_facts: no
  tasks:
    - name: Create a file
      file:
        path: /home/user/sample.txt
        state: touch
```

**Run Command:**

ansible-playbook -i inventory.ini create_file.yml

**Output:**

TASK Create a file

changed true

**Verification:**

ls /home/user

**Verification Output:**

sample.txt

**Explanation:**

file module is used to create file

state touch creates file if it does not exist

---

**Level 2b: Create Multiple Directories using Loop**

**Objective:**

To create multiple directories using single task

**Playbook File:** nano multi_dir.yml

```
- name: Create multiple directories
  hosts: local
  gather_facts: no
  tasks:
    - name: Create directories
      file:
        path: "/home/user/{{ item }}"
        state: directory
      with_items:
```

```
    - dir1
    - dir2
    - dir3
```

**Run Command:**
ansible-playbook -i inventory.ini multi_dir.yml

**Output:**
TASK Create directories
changed true

**Verification:**
ls /home/user

**Verification Output:**
dir1
dir2
dir3

**Explanation:**
with_items loop runs task multiple times
Each item creates one directory

---

**Result:**
Playbooks were successfully executed and files and multiple directories were created using Ansible

## Lab Sheet 9: Selenium IDE Basics

---

**Aim:**
To understand Selenium IDE, install it, and create a basic test case including login testing

---

**Requirements:**
Tool: Selenium IDE
Browser: Google Chrome or Mozilla Firefox

---

**Objective:**
To understand basic automation testing using Selenium IDE

**Explanation:**
Selenium IDE is used to automate browser actions like clicking, typing, and testing web applications
It works as a browser extension and records user actions

---

**Level 2: Selenium IDE Download and Install**

**Objective:**
To install Selenium IDE

**Steps:**

**Step 1:** Open browser

**Step 2:** Search Selenium IDE extension

**Step 3:** Install from browser store

**Step 4:** Click on Selenium IDE icon after installation

**Step 5:** Choose create new project

---

**Level 2: Selenium IDE First Test Case**

**Objective:**
To create first test case

**Steps:**

**Step 1:** Open Selenium IDE

**Step 2:** Click create new project

**Step 3:** Enter project name

**Step 4:** Click record new test

**Step 5:** Enter website URL

https://practicetestautomation.com/practice-test-login/

**Step 6:** Perform actions like clicking and typing

**Step 7:** Stop recording

**Step 8:** Save test case

---

**Output:**
Test case is recorded and displayed in Selenium IDE

---

**Level 2: Login Test using Selenium IDE**

**Objective:**
To automate login process

**Steps:**

**Step 1:** Start recording

**Step 2:** Open login page

**Step 3:** Enter username

**Step 4:** Enter password

**Step 5:** Click login button

**Step 6:** Stop recording

---

**Output:**
All login steps are recorded as commands

---

**Common Commands Used in Selenium IDE**

**open:** Opens a webpage
Example open https example com

**click:** Clicks on button or link
Example click login button

**type:** Enters text into input field
Example type username

**assert:** Checks condition
Example assert text

---

**Explanation:**
Commands are instructions given to browser
Each action is stored as a command in test case

---

**Result:**
Selenium IDE was successfully installed and test cases including login test were created and executed

# Lab Sheet 10: Selenium WebDriver with Chrome

**Aim:**
To install Selenium and write scripts to open Google and verify title and redirection

---

**Requirements:**
Operating System: Linux Mint
Tool: Selenium WebDriver
Browser: Google Chrome
Driver: ChromeDriver
Python installed

---

**Level 1: Open google.co.in using Chrome**

---

**Step 1: Check Python Installation**
python3 --version

---

**Step 2: Install Virtual Environment**
sudo apt install python3-venv

---

**Step 3: Create Virtual Environment**
python3 -m venv myenv

---

**Step 4: Activate Environment**
source myenv/bin/activate

---

**Step 5: Install Selenium**
pip install selenium

---

**Step 6: Create Python File**
nano test.py

---

**Step 7: Write Code**

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("https://www.google.co.in")
input("Press Enter to close")
driver.quit()
```

---

**Step 8: Run Script**
python test.py

---

**Output:**
Google homepage opens in Chrome browser

---

**Level 2: Verify Title and Redirection**

---

**Step 1: Edit File**
nano test.py

---

**Step 2: Write Code**

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("https://www.google.com")
print("Title:", driver.title)
print("URL:", driver.current_url)
if "Google" in driver.title:
    print("Title verification passed")
else:
    print("Title verification failed")
if "google.co.in" in driver.current_url:
    print("Redirection passed")
else:
    print("Redirection failed")
input("Press Enter to close")
driver.quit()
```

---

**Step 3: Run Script**
python test.py

---

**Output:**
Title: Google
URL: https://www.google.co.in
Title verification passed
Redirection passed

---

**Explanation:**
webdriver launches browser

get method opens URL
title checks page title
current_url checks redirection

---

**Result:**
Selenium was successfully installed using virtual environment and scripts were executed to verify Google title and redirection

## Lab Sheet 11: Selenium WebDriver using Firefox (Linux )

**Aim:**
To open Google using Firefox browser and create browser instance based on browser name

---

**Requirements:**
Operating System: Linux
Tool: Selenium WebDriver
Browser: Firefox
Driver: Geckodriver
Python installed

---

### Level 1: Open google.co.in using Firefox

---

### Step 1: Open Terminal

---

### Step 2: Go to Downloads Folder
cd ~/Downloads

---

### Step 3: Download Geckodriver

wget
https://github.com/mozilla/geckodriver/releases/download/v0.34.0/geckodriver-v0.34.0-linux64.tar.gz

---

**Step 4: Extract File**
tar -xvzf geckodriver-v0.34.0-linux64.tar.gz

---

**Step 5: Verify Extraction**
ls

Output should show:
geckodriver

---

**Step 6: Move Geckodriver to System Path**
sudo mv geckodriver /usr/local/bin/

---

**Step 7: Give Permission**
sudo chmod +x /usr/local/bin/geckodriver

---

**Step 8: Verify Installation**
which geckodriver

Output:
/usr/local/bin/geckodriver

---

**Step 9: Create Python File**
nano test.py

---

**Step 10: Write Code**

```
from selenium import webdriver

driver = webdriver.Firefox()
driver.get("https://www.google.co.in")

input("Press Enter to close")
driver.quit()
```

---

**Step 11: Activate Virtual Environment**

```
source myenv/bin/activate
```

---

**Step 12: Run Script**

```
python test.py
```

---

**Output:**

Firefox browser opens and Google homepage is displayed

---

## Level 2: Create Browser Instance based on Browser Name

---

**Step 1: Create Python File**

```
nano browser_test.py
```

---

**Step 2: Write Code**

```
from selenium import webdriver
browser = input("Enter browser name chrome or firefox ")
if browser == "chrome":
 driver = webdriver.Chrome()
elif browser == "firefox":
 driver = webdriver.Firefox()
else:
 print("Invalid browser")
 exit()
driver.get("https://www.google.com")
input("Press Enter to close")
driver.quit()
```

---

**Step 3: Run Script**

python browser_test.py

---

**Step 4: Enter Input**

firefox

---

**Output:**

Firefox browser opens successfully

---

**Explanation:**

Geckodriver is required to connect Selenium with Firefox

webdriver.Firefox is used to launch Firefox browser
Program selects browser based on user input

---

**Result:**
Firefox browser was successfully automated and browser selection program was executed using Selenium WebDriver

## Lab Sheet 12: Selenium WebDriver Browser Control and Listbox Handling

**Aim:**
To close browser without using quit method and to search for an option in listbox using Selenium

---

**Requirements:**
Operating System: Linux
Tool: Selenium WebDriver
Browser: Firefox or Chrome
Driver: Geckodriver or ChromeDriver
Python installed

---

### Level 1: Close Browser without using quit()

---

**Objective:**
To close browser without using quit method

---

**Step 1: Create Python File**
nano close_browser.py

---

**Step 2: Write Code**

```
from selenium import webdriver

driver = webdriver.Firefox()
driver.get("https://www.google.com")

input("Press Enter to close browser")

driver.close()
```

---

**Step 3: Run Script**

```
python close_browser.py
```

---

**Output:**

Browser opens Google page and closes after pressing Enter

---

**Explanation:**

close method closes current browser window
quit method closes all browser windows
Here we used close instead of quit

---

**Level 2: Search for Specified Option in Listbox**

---

**Objective:**

To select and search an option from dropdown listbox

---

**Step 1: Create Python File**

```
nano listbox.py
```

---

**Step 2: Write Code**

```
from selenium import webdriver
from selenium.webdriver.support.ui import Select

driver = webdriver.Firefox()
driver.get("https://www.w3schools.com/tags/tryit.asp?filename=tryhtml_select")

driver.switch_to.frame("iframeResult")

dropdown = Select(driver.find_element("tag name", "select"))

dropdown.select_by_visible_text("Saab")

input("Press Enter to close")
driver.close()
```

---

**Step 3: Run Script**

```
python listbox.py
```

---

**Output:**

Dropdown list opens and option Saab is selected

---

**Explanation:**

Select class is used to handle dropdown
select_by_visible_text selects option by name
switch_to.frame is used because dropdown is inside frame

---

**Result:**

Browser was successfully controlled without quit method and listbox option was selected using Selenium

---

**Important Note:**

close closes only one tab while quit closes entire browser session